ChatGPT

The 100% Cost for 10% Use Problem (Software Bloat and Inefficiency)

Most organizations today are paying for far more software functionality than they actually use. Studies show that users typically utilize only a small fraction of features in major software packages – often around 10% of the available functionality [1]. Yet they pay 100% of the license or subscription cost. For example, in off-the-shelf applications like Microsoft Word, the majority of features sit idle while only the core 10% are used daily [1]. This feature bloat is not just theoretical: over half of IT professionals report their organizations pay for SaaS features that their teams *never* use [2]. In practice, companies are spending money and complexity budget on functions that bring no value to their workflows.

This imbalance leads to significant waste and frustration. One report found that IT teams lose an average of 7+ hours per week dealing with bloated, overly complicated software [3] [4]. That translates to billions in lost productivity. Nearly 90% of IT pros say they have frustrations with their company's software, citing slow performance, inflexibility, and having to juggle multiple tools to accomplish tasks [5] [6]. In essence, **organizations are paying for 100% of a product but only getting 10% of the benefit** – a poor deal for customers. As one industry expert put it, "more software isn't always better," and CIOs realize it's time to *"break the doom cycle of bloatware"* in order to help employees succeed [7] [8].

Beyond wasted money, feature bloat creates *vendor lock-in*. Big software vendors intentionally keep adding features to widen their product's scope, creating a *moat* of complexity that deters switching [9]. Even if a customer only needs a few core features, they hesitate to switch to a simpler alternative unless it matches the full breadth of the bloated incumbent [10]. In other words, **unused features become switching barriers** – enterprises fear that one day they *might* need that advanced feature, so they stick with the heavy, expensive solution "just in case" [11] [9]. Vendors capitalize on this psychology by bundling a comprehensive feature set (often far beyond any single customer's needs) and charging accordingly. Cloud platforms are a prime example: AWS offers over 200 services while a typical customer uses fewer than 10, yet the sheer breadth locks customers in because competitors can't match the full catalog [9]. This **"feature gravity"** is a deliberate strategy to keep users paying for the whole, even when they only use a part.

Finally, customers have little agency to change this dynamic. Most companies cannot influence the roadmap of a large SaaS or enterprise software provider – they are just one voice among thousands of customers. In fact, *7 out of 10* IT professionals hesitate to even give feedback on their bloated software, often because they feel their input will be ignored or they don't want to be seen as complainers [12]. This lack of voice means that if the 10% of features customers do use are missing something, they usually have to wait (often in vain) for the vendor to listen. The result is widespread frustration: nearly **89% of IT pros are not satisfied** with their company's software, and 71% think their organization would benefit from simpler, more streamlined software [4] [13]. In short, there is a growing recognition that the status quo – paying full price for bloated products and being stuck with features you don't need – is unsustainable.

The Emergence of Focused “10% Solutions”

A promising solution to this problem is now emerging: **highly focused software services that deliver exactly the 10% of features users actually need, at roughly 10% of the cost**. Instead of one-size-fits-all products loaded with bells and whistles, these new solutions are pared-down, purpose-built tools. The core idea is simple: if customers are only using a small subset of features from a complex product, why not build a *new* product that consists *only* of that subset? Users would then pay only for what they truly use (say 10% of the original price) and in return get 100% of the utility they care about. This flips the equation to **“10% cost for 100% of your use case.”**

This pattern is becoming increasingly viable thanks to modern development practices and open-source ecosystems. In the past, creating a custom version of a large software suite was often impractical – it could take large teams and years of work to replicate even a fraction of a mature product’s functionality. Today, however, several factors reduce the effort needed:

- **Open-Source Foundations:** Virtually every domain now has open-source libraries or platforms that cover the basics. A small team can start with these ready-made components (the “global commons” of software) instead of reinventing the wheel [14] [15]. For example, if you want to build a lightweight CRM with just the essential features, you might combine an open-source database, a web framework, and existing open-source CRM modules – dramatically cutting development time. Open source provides a *head start* by offering the building blocks for 80-90% of any common software functionality.

- **Low-Code and Code Generation Tools:** New AI-powered development tools (including large language models, or LLMs) can **kickstart the creation** of software components that previously would have taken significant manual coding. Developers can now use AI assistants to generate boilerplate code, set up basic UIs, or even scaffold entire modules in a few hours. This means a prototype of a targeted “10% feature” product can be created rapidly, then iterated to production quality. (We will dive deeper into AI’s role in a later section.)

- **Microservices and APIs:** Modern software architectures make it easier to isolate just the needed functionality. Instead of monolithic suites, we can build microservices or plugins that perform specific tasks. This modular approach aligns perfectly with the idea of focused solutions – a team can develop a single-service product that integrates with others as needed, rather than a giant all-in-one platform.

Crucially, these focused solutions often leverage **open-source licensing and collaboration**. Many of the emerging “10% solutions” are being released as open-source or built on open-source cores. This has two major benefits:

1. **Collective Improvement:** If multiple organizations all need that 10% functionality, they can share the cost of improving it. An open-source project can aggregate contributions from many local implementations, ensuring that even a small regional software gets improvements and features added by a broader community. This community model means each local solution doesn’t have to invent everything in isolation – they can pull from a common pool of code (global or European open-source projects) and then add their specific customizations [14] [16]. Over time, the best ideas from each deployment can merge upstream, resulting in a robust core that everyone benefits from.

2. **No Vendor Lock-In:** By using open-source, these solutions avoid the new boss same as the old boss scenario. Users gain full control – they can host it themselves, audit the code, extend it, or switch providers if needed. There's no opaque black-box holding their data hostage. This aligns with a growing demand for **digital sovereignty**: organizations want to run software on their own terms without being at the mercy of a distant vendor's pricing or policies [17] [18]. Focused open-source replacements for SaaS give exactly that freedom.

We are essentially witnessing an **unbundling** of big software. Where a legacy vendor offered a giant suite to do *everything*, nimble new companies (and open-source projects) are now picking off one narrow slice at a time – the slice that users actually use – and doing it better. For example, consider a hospitality business in a specific region that currently uses a global SaaS platform for reservations and customer management. Perhaps 90% of that platform's features are meant for large international hotel chains and go unused, while 10% cover local boutique needs. A local startup could build a slim, tailored reservation system focusing purely on what local hospitality providers require, with a simpler interface and none of the excess. The local business gets a solution that **covers 100% of their daily needs at a fraction of the cost** of the bloated SaaS subscription. Meanwhile, they also gain the ability to request tweaks or new features from the local provider – something impossible with a big global vendor.

Notably, this trend also encompasses **open-source alternatives to common SaaS tools** that have matured in recent years. From office suites to design software, open-source options often cover *"most everyday use cases"* with far lower cost and complexity [19] [20]. For instance, LibreOffice provides the core 10% of features that most users need from Microsoft Office (word processing, spreadsheets, presentations) but without the licensing fees – as evidenced by many institutions successfully switching to LibreOffice and saving money [21] [22]. Similarly, tools like GIMP or Inkscape offer the essential subset of Photoshop/Illustrator capabilities needed by the average designer, and Penpot or Logseq target the key features of Figma or Notion, respectively [19] [20]. These community-driven projects focus on *what users actually use*, and by doing so they dramatically lower the barrier to adoption (no endless feature toggles to confuse users, and usually no cost aside from optional support).

In summary, **focused "10% solutions" aim to deliver precisely the functionality users value, and nothing more**. They challenge the old assumption that software must be one giant package. Instead, they prove that *leaner is better*: simpler apps are easier to use, have fewer bugs, cost less to run, and can be improved more quickly in response to user feedback. This paradigm shift – from paying 100% for 10% use, to paying 10% for 100% use – is poised to reshape software economics. It also opens the door to a new ecosystem of companies that specialize in these niche-but-valuable replacements. Interestingly, nowhere might this model be more powerful than in **Europe**, where diverse local needs and an open-source ethos create fertile ground for such focused solutions.

Europe's Distributed Advantage and Open-Source Opportunity

Europe, perhaps more than any other region, stands to *benefit from and lead* this shift toward customized, right-sized software. One of Europe's traditional challenges in tech – its fragmentation into many countries, languages, and markets – can be turned into a strategic advantage in this context. Here's why:

1. One Size Doesn't Fit Europe: Europe is not a monolithic market but a mosaic of local needs. A tourism business in the Algarve region of Portugal has different requirements (and perhaps operates in a different language) than a tourism business in rural Poland or the south of France. Large global

software vendors often provide a one-size-fits-all product that leaves these niche needs under-served. This creates opportunity for **local software companies to build tailored solutions** for local industries, cultures, and regulations. Europe's multicultural diversity *demands* more customization – and thus it naturally creates thousands of micro-markets where focused 10% solutions could thrive. Embracing this model could become a *"European strategic advantage,"* capitalizing on what makes Europe unique (distributed, multicultural, multi-lingual environments).

2. Vibrant Ecosystem of Small Tech Firms: Unlike the U.S. or China, which have a few dominant tech giants, Europe's tech landscape is characterized by a *vibrant ecosystem of smaller companies and startups* [23]. These smaller firms are agile and closer to their local customers. They are well positioned to build the bespoke solutions for a specific region or sector. Open source and collaborative development allow them to punch above their weight: by pooling efforts on common needs, European SMEs can together create interoperable software stacks that rival those of big vendors [14] [16]. **Open source is the great equalizer** here – it enables many organizations to work together on shared foundations, which is exactly how a collection of SMEs can compete with tech giants [15]. As the Open Source Initiative notes, this model of community-driven development plays to Europe's strengths and allows catching up without each company reinventing the wheel [15].

3. Digital Sovereignty and Local Control: Europe in recent years has prioritized *digital sovereignty* – ensuring that critical digital infrastructure is under local control and not subject to external leverage or surveillance [24] [25]. Focused open-source solutions are perfectly aligned with this goal. Because they can be self-hosted and modified, they keep data and control within the country or region. The code can be audited for compliance with EU regulations (privacy, security, etc.) [14]. And there's no risk of a foreign company suddenly cutting off access or changing terms arbitrarily [25] [17]. In fact, we've already seen European public institutions opting for open-source alternatives to regain autonomy: for example, Denmark's Ministry of Digitalisation decided to migrate from Microsoft 365 to LibreOffice, and Germany's state of Schleswig-Holstein is moving 30,000 workstations off Microsoft products in favor of open solutions [26]. These moves show a clear preference to **stop overpaying big foreign vendors for bloated software and instead invest in sovereign tech**. By developing local software that covers the needed 10% of features, European organizations can free themselves from being "held hostage" to distant software monopolies [24].

4. Local Economic Development: An often underappreciated benefit of embracing open, focused software is the boost to local economies. Instead of sending large license fees overseas to a multinational software vendor, that money can pay local developers and service providers. *Open source stimulates local economies* – municipalities and businesses can contract local IT firms to implement and customize open-source solutions, keeping talent and money in the region [27]. For example, the city of Turin trained its staff on Linux and LibreOffice, building internal expertise and fostering a skilled local open-source community [28]. Toulouse, France's 4th largest city, not only saved over €1 million in three years by switching its 10,000 employees to LibreOffice, but explicitly noted that "the open model promotes economic development and employment in the region" [21]. In other words, adopting lean open-source tools creates a virtuous cycle: **cost savings, reinvestment in local talent, and new startups** that grow to service the software. Europe's policy-makers have recognized this; the European Commission actively encourages creating a strong open-source ecosystem of local companies that build on the global commons and provide local support and services [29]. This is seen as key to competitiveness and reducing dependency.

5. Collaborative Culture and Public Sector Support: Europe has a culture of collaboration between public and private sectors around open technology. We see cities and regions teaming up to co-develop open-source solutions for common needs [30] [31]. Projects like Decidim (participatory democracy platform started in Barcelona) or the OS2 community in Denmark show that cross-border sharing of

software is feasible and encouraged [32]. This means a focused solution created for one city/region can be adopted by another with similar needs, with improvements flowing back – a powerful network effect, but in the open-source sense. The EU's open-source observatory (OSOR) documents many such case studies and actively promotes re-use of solutions across regions [32] [33]. Furthermore, procurement policies are slowly being reformed to favor open-source options and to fund the maintainers (so that the money goes into improving the code, not just paying middlemen) [34] [35]. All this top-down support complements the bottom-up innovation of startups. It signals that **Europe is embracing "open" as the way forward** – which fits perfectly with the rise of these lean, open-source alternatives.

In summary, Europe's distributed nature is not a bug but a feature in this new paradigm. It *necessitates* bespoke solutions – and by embracing open-source and fostering local tech ecosystems, Europe can turn that into a competitive edge. By January 2026, this trend is visibly accelerating: European companies and governments are increasingly looking for simpler software that they can control and shape. In the next sections, we'll explore how new technologies like AI help make these focused solutions feasible, and why this model implies not a decrease but an **increase in demand for skilled developers and engineers** in Europe.

AI and LLMs: Catalysts for Localized Development

One reason the "10% solution" revolution is happening now (and not a decade ago) is the advent of advanced development tools powered by AI, particularly **large language models (LLMs)**. These AI coding assistants have become *game changers* for software development, effectively **democratizing** the ability to create and customize software. This is critical for small local companies or even individual entrepreneurs who want to build a tailored product for a niche market. In the past, even a small clone of a larger product required a team of developers; today, an **AI-assisted solo developer or tiny team can achieve the same in a fraction of the time** [36] [37].

Here's how LLM-driven tools are empowering these new European software endeavors:

- **Rapid Prototyping & Coding:** Modern IDEs integrated with LLMs (like GitHub Copilot, Cursor, etc.) allow a developer to generate substantial chunks of code from high-level prompts. For example, a developer described asking an AI assistant (Cursor IDE) to implement a new feature – within seconds the AI produced a complete code file, complete with configuration parameters and integration instructions, all following the project's coding style [38] [39]. This kind of acceleration means that a lone developer can iterate prototypes extremely quickly. What might have taken weeks of trial-and-error coding can now be done in days. For a local startup trying to replicate just the essential 10% of a big system, this is a huge boost – they can **kickstart development** of each component with AI suggestions, then refine from there.

- **Multi-Role Automation:** LLMs don't just write code – they can also generate documentation, tests, and even help with design. In an AI-first development workflow, one or two engineers can effectively wear all the hats (product manager, coder, tester, even UX designer) with AI helpers taking on much of the grunt work [36] [40]. As a recent analysis noted, an AI-"supercharged" developer can handle architecture, implementation, and low-level design solo, something that used to require a team [36]. Routine tasks like writing unit tests or translating user stories into tasks can be offloaded to AI. This means even a small local company can maintain a full-stack, complex application without needing dozens of specialists – *the AI acts as force multiplier*. A one-

person team using AI can deliver a functional product to a client, which is an ideal scenario for serving a small regional market efficiently [41] [36].

- **Local Language and Domain Customization:** For European contexts, LLMs can also help with localization and domain-specific needs. Need your software's UI in Catalan or Polish? An LLM can assist in translating and generating locale-specific content. Need to incorporate local regulations (say, GDPR compliance features, or a tax calculation module specific to a country)? An LLM can quickly generate code snippets or logic patterns based on legal texts and examples. Essentially, AI can bridge knowledge gaps, allowing a small team to incorporate highly specific features that normally would require niche expertise. This is particularly helpful in Europe, where each region might have its own rules and language – AI helps level the playing field by providing on-demand knowledge and even generating code conforming to those local requirements.

- **Reduced Barrier to Entry for New Developers:** The presence of AI assistants makes software creation more accessible to those who are not seasoned engineers. A local entrepreneur or domain expert (say a person from the tourism industry in Algarve) could leverage no-code/low-code tools augmented with AI to build a prototype of the solution they envision, without writing every line from scratch. They might then hire a few junior developers to polish it. The point is that **LLMs lower the initial skill and effort required to get a software project off the ground**. This can activate a whole new wave of local innovation – people with great ideas but limited coding experience can now attempt to build software tailored to their community's needs, with AI as a tutor and assistant.

It's important to stress that AI is a catalyst, not a replacement for human developers. The role of the developer is evolving rather than shrinking. As AI handles repetitive tasks, human engineers can focus more on **good engineering practices** – designing architecture, ensuring security, and refining user experience. In fact, small teams must be careful: AI-generated code is not magically perfect. Studies have shown that a significant portion of AI-generated code may contain errors or security vulnerabilities (one study found about 32% of code suggestions from GitHub Copilot had potential security issues) [42]. Therefore, the human developer's job is crucial in **reviewing, testing, and hardening** the output. As the user of our hypothetical scenario wisely noted, *"everything should arrive at good engineering... code that is maintainable, understandable, and not about black magic."* The AI can draft the blueprint, but engineers must ensure the final structure is sound.

In practice, we are seeing the rise of the **"LLM Engineer"** – a developer who is adept at collaborating with AI tools to produce quality software [36] [43]. This new skillset involves knowing how to prompt the AI effectively, how to verify and refine its outputs, and how to blend AI suggestions with one's own code. For Europe, building up a workforce of such LLM-empowered developers could be transformative. It addresses the talent shortage (each developer is more productive) and makes it feasible to tackle the myriad small-scale projects (for local governments, SMEs, etc.) that were previously uneconomical to build. Instead of importing a generic product from a big vendor, a local team can now *build their own* solution with modest effort.

In conclusion, AI and LLM workflows significantly reduce the development time and resources needed to create custom software, **making the "10% approach" practical on a wide scale**. A small company in Lisbon or Ljubljana can clone the necessary 10% of a large system and tailor it to local needs, with an efficiency that was unimaginable just a few years ago. This technology trend strongly reinforces Europe's push towards local, sovereign solutions – it's the wind in the sails, enabling Europe's many small tech boats to move faster and compete on their home turf.

From Prototype to Production: Emphasizing Quality Engineering

While the combination of open source building blocks and AI-generated code can get a project off the ground quickly, success in these focused solutions will ultimately hinge on **software quality and maintainability**. If a multitude of new small-scale applications are to replace big established software, they must earn users' trust by being reliable, secure, and well-supported. In other words, it's not enough to *"vibe-code"* an app that works only in demo scenarios – the goal is to arrive at **professional-grade engineering**. Europe's opportunity in this movement will falter if the resulting software is shoddy or unsustainable. Therefore, a strong emphasis is being placed on good engineering practices atop the AI-generated scaffolding:

- **Clean, Understandable Codebases:** The new solutions should strive for simplicity not just in features but in code structure. Because many of these projects will be open-source, anyone should be able to read and grasp the code. Using established frameworks, clear documentation, and adhering to style standards are important. If a local company in one region builds a solution, a different team elsewhere might later adopt or fork it – they'll need to navigate the code easily. Ensuring that AI helpers are directed to produce *clean code* (for example, by prompting for well-commented functions and proper modularization) is part of the LLM engineer's task.

- **Maintainability and Updates:** One criticism of having many niche software pieces is potential duplication of effort or divergence. This is mitigated by open source licensing (sharing improvements) and by good architecture. If each focused solution cleanly encapsulates its domain, maintaining it over time is feasible for a small team. Additionally, because these tools are smaller in scope, updating them for new requirements or technology changes can be faster than in a monolithic suite. The key is disciplined project management: using version control, continuous integration, and automated testing from the start – all the things that ensure changes don't break the system. As mentioned earlier, AI can generate a lot of unit tests and even assist in setting up CI pipelines, which new projects should leverage for long-term health [44] [45].

- **Security and Trustworthiness:** Software that is intended to run in a local or sovereign context must be secure. While big vendors have whole security teams, smaller firms must not skimp on this aspect. Again, open source plays a helpful role (many eyes can inspect the code for vulnerabilities). There are also security-focused open-source tools that can be integrated (for example, libraries for encryption, authentication, etc., that are widely vetted). European projects might collaborate on security audits for the common components they use. It's also worth noting that **transparency is a selling point** – EU users (especially public sector) often prefer an open solution they can inspect over a proprietary one that just asks for trust [14]. By baking in security and privacy-by-design – something European regulations strongly emphasize – these small solutions can actually outshine large competitors on compliance and trust.

- **User-Centric Design:** One major advantage of focused software is reducing user frustration. To fulfill that promise, the creators of these new tools must engage closely with their users (often they *are* the users or live in the same community). This allows rapid feedback loops: if something isn't working or a feature is missing, the local developer can add it in the next release. Contrast this with big vendors where feature requests vanish into a black hole. By prioritizing the 100% of features *users actually care about*, these solutions inherently deliver better UX. But developers must resist feature creep – always circle back to, "does this serve the majority of my users' daily

workflow?" Adopting methodologies like agile (with quick iterations) or even lean UX research within the local context will ensure the product stays sharply aligned with user needs rather than accumulating unnecessary complexity.

- **Interoperability and Standards:** To avoid creating a new kind of lock-in (just at a smaller scale), European focused solutions should adhere to open standards for data formats and integration. One historical moat of proprietary software was using closed file formats or APIs to trap users. Today, with the push for interoperability (a theme in EU digital strategy), a local solution should make it easy to import/export data in standard formats (for example, using CSV, JSON, or industry-specific standards) and to plug into other software via well-documented REST or GraphQL APIs. By doing so, even if one small tool doesn't cover everything, it can work in concert with others. A tourism CRM might export data to a standard accounting system, or a local healthcare app might integrate with national health records via open protocols. This ensures that each focused tool slots into a bigger ecosystem rather than becoming an island. And since many integrations are common needs, open-source connectors or middleware can be shared across regions.

The overarching point is that **quality matters**. Fast development via AI is wonderful, but not at the expense of robustness. Encouragingly, the need for quality is well-understood by proponents of this movement. They argue that far from reducing the need for developers, these trends will **increase demand for skilled engineers** – because we will be building *more* software (highly customized for each locale and purpose) than ever before, and we need professionals to architect, polish, and maintain all those systems. AI and open source reduce the cost barrier and drudgery, but human insight is needed for the final 10% that turns a prototype into a polished product. As one CIO noted, the goal is to prioritize *productivity* and effectiveness of software, not just add features for features' sake [8]. Achieving that requires thoughtful design and engineering discipline.

An Emerging Open-Source Business Model

The shift toward bespoke, minimal software solutions is also giving rise to what could be **the next business model for open source in Europe**. Traditionally, open-source software has been funded in various ways (support contracts, dual licensing, open-core models, etc.), and many open-source startups sought to scale globally to compete with proprietary giants. In this new paradigm, however, we envision **a network of regionally focused open-source companies**, each sustainably serving their community's needs. Here's how the business model tends to look:

- **Service-Oriented Revenue:** Rather than selling software licenses, these companies often earn revenue through services – customization, deployment, hosting, and support. For example, a small firm might offer a SaaS-hosted version of the open-source tool for the local market at a fair price, or charge for on-premise setup and training for clients like local government offices or SMBs. Because the software is open source, the *value* the company provides is expertise and service. This aligns with European preferences in public procurement, which increasingly favor paying local providers to adapt and support open solutions, rather than paying license fees to foreign vendors [34] [35]. It keeps money circulating in the local economy and builds long-term customer relationships.

- **Consortium and Co-op Models:** We may see clusters of stakeholders co-fund development of the open-source base. For instance, a group of tourist boards across several regions could pool funds to develop the core features of a tourism management app (thus each pays a fraction of the cost for 100% of what they need). Individual companies then compete or collaborate on

providing the best localized service on top of that core. This “cooperative competition” model fits well with European business culture, which often emphasizes collaboration (e.g., consortiums in EU research projects) even among competing entities for pre-competitive development. Open source enables such arrangements because everyone can contribute to and benefit from the shared core platform.

- **Scaling by Horizontal Expansion:** Unlike a traditional startup that must “vertically” expand by adding more features to grow revenue (often leading to bloat), an open-source company can expand horizontally by tackling adjacent niches. For example, a company that built a lightweight CRM for local hotels might use a similar framework to build a booking system for local restaurants. They don’t bloat one product to sell more – they spawn a *suite of focused tools* that share components. This way, each product stays lean, but the company builds expertise and reusable tech that gives it economies of scale internally. Essentially, the business grows not by making one product do everything, but by doing many small products well. Open source libraries and internal reuse make this efficient. In Europe, where markets are segmented, a company might replicate its success from one country to another – exporting the model while translating and tweaking it for each locale.

- **Community and Subscription Hybrid:** Some open-source companies are adopting models where the core software is free (and community-maintained) but value-added services or cloud convenience are paid. European customers who have limited budgets can stick to the free self-hosted version (maybe maintained by an in-house IT team), whereas others pay for the managed version. The key difference from large vendors is the *lack of lock-in*: if the vendor’s pricing isn’t justified, the customer can revert to self-hosting since it’s open source. This forces the open-source provider to truly add value to earn subscriptions (e.g., by providing hassle-free updates, extra cloud features, or premium support). It’s a healthier, more competitive dynamic that keeps the focus on user satisfaction.

Importantly, European regulatory trends could accelerate this business model. The EU is actively exploring ways to **make open-source development sustainable** as part of its digital sovereignty agenda. There are proposals for things like a “Buy Open Source” policy, where public sectors prioritize open solutions and even contribute back (financially or with code) to those projects [46] [47]. If governments direct more contracts to open-source maintainers and local integrators, it creates a steady revenue stream to grow these focused solutions [35] [48]. This might reverse the old problem Dries Buytaert noted, where public money flowed *around* open-source (to integrators who didn’t contribute) instead of *into* it [34] [35]. In the new model, the local companies building the 10% solutions *are often the maintainers or major contributors* of the open-source project itself, so funding them equates to funding the project’s improvement. Over time, this strengthens the product for all users, creating positive feedback.

The end result could be a **rich tapestry of open-source companies across Europe**, each deeply embedded in their local context, but loosely federated by shared open-source projects and standards. Some will provide core infrastructure or platforms (the building blocks), some will specialize in domain-specific logic, and others will focus on user-facing customization and support. Together they form a supply chain of software creation that is distributed yet cooperative. It’s a different vision of technology industry growth – one that doesn’t require spawning the next Silicon Valley juggernaut, but rather nurturing *thousands of smaller players* adding up to a big impact. This aligns with Europe’s economic structure and policy goals, leveraging the power of open source as a collective commons to reduce reliance on a few mega-vendors [49] [29].

Conclusion: A Lean, Sovereign, and User-Centric Future

As of January 2026, the pieces are coming together for a fundamental shift in how software is developed, sold, and consumed – a shift particularly favorable to Europe's values and needs. The old model of monolithic software suites, where users pay for countless features they don't use, is increasingly seen as broken. In its place, a new paradigm is rising: **lean, customized solutions that put the user's needs front and center, with pricing to match actual usage and value delivered**.

This new paradigm is underpinned by open source collaboration, ensuring that no single entity controls the technology and that improvements are shared. It embraces Europe's diversity by allowing localization and specialization, turning what was once viewed as fragmentation into an engine of innovation. By fostering local tech talent and businesses, it bolsters regional economies and digital sovereignty – Europeans building for Europe, on a foundation accessible to all [50] [27]. And with the aid of AI tools and modern practices, even the smallest teams can create high-quality software tailored to niche demands, without compromising on engineering excellence.

Of course, challenges remain. Larger incumbent vendors will not sit idle; they may respond by offering more modular pricing or acquiring promising open-source tools. There is also the question of support and longevity – will all these small solutions have the staying power of established products? The answer will depend on the strength of the communities and companies behind them. Early signs are encouraging: when multiple stakeholders have a shared interest (say, a consortium of cities using a piece of software), that greatly increases the chance the software will be maintained long-term, because it's mission-critical for many, not a side project. Moreover, the European Commission and national governments are aware of the need to support the open-source ecosystem, which could mean more direct funding, bug bounties, and institutional adoption that gives these projects a solid user base [51] [29].

In essence, the stage is set for Europe to **change the script** in tech: from being a consumer of bloated foreign software to a co-creator of slim, effective, and open solutions. Businesses and public services can regain agency – if a feature is missing, they can build it or sponsor its development, rather than pleading with a distant vendor. Users can regain simplicity – using tools that do exactly what they need and nothing extraneous, improving productivity and even well-being (no more fighting unwieldy interfaces or paying for unused licenses). And developers have a new frontier – far from AI replacing them, it is enabling them to multiply their impact, meaning *more* developers will be needed to fulfill all the custom requests and to enforce quality across the board.

The pattern of "100% pay for 10% use" belongs to a bygone era. The future is **paying 10% for the 100% you actually use**, and reinvesting the other 90% of resources into making that software better, or addressing other needs. It's a future where software is right-sized, users are empowered, and local innovation thrives. Europe is poised to lead this charge, showcasing how open source and distributed creativity can out-innovate even the mightiest of centralized tech empires. The transformation is already underway – and the coming years could very well see this approach become *the norm* rather than the exception, delivering technology that is **simpler, fairer, and truly built-for-purpose** for everyone.

Sources:

- Freshworks, *State of Workplace Technology: Bloatware* report (Aug 2022) – findings on software bloat and unused features [2] [12].

- Netguru Blog – *Custom Software vs Off-the-Shelf* (May 2025) – notes that users use ~10% of features in tools like MS Word [1].
- SoftwareSeni – *Feature Paradox* analysis – on how unused features create lock-in and moats [10] [9].
- Thierry Carrez (OSI) – *Open Source: A Global Commons to Enable Digital Sovereignty* (Nov 2025) – Europe's open-source advantage and need for local companies [16] [29].
- Dries Buytaert – *Funding Open Source for Digital Sovereignty* (Jan 2026) – examples of European moves to open source (ICC, Denmark, Germany) and procurement reform [26] [46].
- Interoperable Europe (OSOR) – *Open Source in Cities and Regions* – benefits of open source for local economies, talent, and case studies (Turin, Toulouse etc.) [27] [21].
- LinkedIn Pulse – *Waterfall 2.0: LLM-Driven Workflows* (Oct 2025) – on how one AI-augmented engineer can fulfill multiple roles in development [36] [37].
- Medium – *"Stop paying forever. Start owning your tools"* (Dec 2025) – trend of open-source alternatives replacing SaaS, reducing costs and giving control [19].

---

[1] Custom Software vs Off-the-Shelf: Hidden Costs & Benefits Revealed
https://www.netguru.com/blog/custom-software-vs-off-the-shelf

[2] [3] [4] [5] [6] [7] [8] [12] [13] Bloatware: IT Pros Waste Time Due to Bloated Applications | APMdigest
https://www.apmdigest.com/bloatware-it-pros-waste-time-due-to-bloated-applications

[9] [10] [11] The Hidden Mathematics of Tech Markets: Network Effects, Power Laws and Platform Dominance - SoftwareSeni
https://www.softwareseni.com/the-hidden-mathematics-of-tech-markets-network-effects-power-laws-and-platform-dominance/

[14] [15] [16] [23] [24] [29] [49] [50] Open Source: A global commons to enable digital sovereignty - Open Source Initiative
https://opensource.org/blog/open-source-a-global-commons-to-enable-digital-sovereignty

[17] [18] [25] [26] [34] [35] [46] [47] [48] [51] Funding Open Source for Digital Sovereignty | Dries Buytaert
https://dri.es/funding-open-source-for-digital-sovereignty

[19] [20] 12 Open-Source Alternatives to Your Paid SaaS Tools You Must Try | by Deep concept | Let's Code Future | Dec, 2025 | Medium
https://medium.com/lets-code-future/12-open-source-alternatives-to-your-paid-saas-tools-you-must-try-604391b02086

[21] [22] Moving to LibreOffice saves T... | Interoperable Europe Portal
https://interoperable-europe.ec.europa.eu/collection/open-source-observatory-osor/news/moving-libreoffice-saves-t

[27] [28] [30] [31] [32] [33] Open Source in Cities and Regions | Interoperable Europe Portal
https://interoperable-europe.ec.europa.eu/collection/open-source-observatory-osor/open-source-cities-and-regions

[36] [37] [38] [39] [40] [41] [42] [43] [44] [45] Waterfall 2.0: LLM-Driven Workflows in Software Development
https://www.linkedin.com/pulse/waterfall-20-llm-driven-workflows-software-georgy-starikov-zrfge